19.2 入站消息规范化

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~560k tokens，输出 ~50k tokens（本章合计）

不同消息平台的消息格式千差万别：Telegram 用 `chat.id` 和 `from.username`，WhatsApp 用 JID（Jabber ID），Discord 用 snowflake ID，Signal 用 E.164 电话号码。OpenClaw 的入站消息管线必须将这些异构格式统一为内部的 `MsgContext` 结构，才能让下游的路由、会话管理和 AI Agent 处理逻辑不需要关心消息来源。

19.2.1 消息格式统一：文本、图片、音频、视频、文件

统一消息上下文：MsgContext

所有通道的入站消息最终都会被转换为 `MsgContext` 结构（定义在 `src/auto-reply/templating.ts`），这是 OpenClaw 内部消息的"通用语言"。`MsgContext` 的核心字段包括：

字段类别	关键字段	说明
通道标识	`Channel`，`AccountId`	来源通道和账号
对话类型	`ChatType`	`"direct"` / `"group"` / `"channel"`
发送者	`From`，`SenderId`，`SenderName`，`SenderUsername`，`SenderE164`	多维度发送者身份
接收者	`To`	目标标识（通常是 bot 自身）
消息内容	`Body`，`BodyForAgent`	原始文本 / 清洗后文本
媒体	`MediaPath`，`MediaUrl`，`MediaType`，`MediaPaths`，`MediaUrls`，`MediaTypes`	附件信息
回复上下文	`ReplyToId`，`ReplyToBody`，`ReplyToIsQuote`	引用的消息
群组信息	`GroupSubject`，`GroupSpace`，`GroupChannel`	群名、空间、频道

字段类别	关键字段	说明
线程	`ThreadLabel` , `MessageThreadId`	线程标签和 ID
会话标签	`ConversationLabel`	用于会话路由的人类可读标签

对话类型归一化

入站消息的第一步是对话类型归一化。不同平台对"私聊"和"群聊"的叫法各不相同：

```
// src/channels/chat-type.ts
export type NormalizedChatType = "direct" | "group" | "channel";

export function normalizeChatType(raw?: string): NormalizedChatType | undefined {
  const value = raw?.trim().toLowerCase();
  if (!value) return undefined;
  if (value === "direct" || value === "dm") return "direct";
  if (value === "group") return "group";
  if (value === "channel") return "channel";
  return undefined;
}
```

平台	原始类型	归一化后
Telegram	`private`	`direct`
Telegram	`group` / `supergroup`	`group`
Telegram	`channel`	`channel`
Discord	DM	`direct`
Discord	`GUILD_TEXT`	`channel`
WhatsApp	私聊	`direct`
WhatsApp	`@g.us` JID	`group`
Slack	DM	`direct`
Slack	`channel` / `group`	`channel`

发送者身份的多维度表示

发送者身份是跨平台差异最大的部分。OpenClaw 用多个互补字段来表示同一个发送者：

```ts
// src/channels/sender-identity.ts（简化）
export function validateSenderIdentity(ctx: MsgContext): string[] {
  const issues: string[] = [];
  const chatType = normalizeChatType(ctx.ChatType);
  const isDirect = chatType === "direct";

  const senderId = ctx.SenderId?.trim() || "";
  const senderName = ctx.SenderName?.trim() || "";
  const senderUsername = ctx.SenderUsername?.trim() || "";
  const senderE164 = ctx.SenderE164?.trim() || "";

  // 群聊中必须有至少一种发送者标识
  if (!isDirect) {
    if (!senderId && !senderName && !senderUsername && !senderE164) {
      issues.push("missing sender identity");
    }
  }

  // E.164 格式校验
  if (senderE164 && !/^\+\d{3,}$/.test(senderE164)) {
    issues.push(`invalid SenderE164: ${senderE164}`);
  }

  // 用户名不应包含 @ 符号
  if (senderUsername && senderUsername.includes("@")) {
    issues.push(`SenderUsername should not include "@"`);
  }

  return issues;
}
```

> **衍生解释：E.164 格式**
>
> E.164 是国际电信联盟（ITU）定义的电话号码格式标准，以 `+` 开头，后跟国家代码和号码，最长 15 位数字。例如 `+8613812345678`（中国手机号）。Signal 和 WhatsApp 都以 E.164 格式标识用户。

不同通道提供不同的字段组合：

通道	SenderId	SenderName	SenderUsername	SenderE164
Telegram	`12345678`	`"张三"`	`"zhangsan"`	—
WhatsApp	`86138...@s.whatsapp.net`	`"张三"`	—	`+8613812345678`
Discord	`1234567890123456`	`"张三"`	`"zhangsan#1234"`	—

通道	SenderId	SenderName	SenderUsername	SenderE164
Signal	—	"张三"	—	+8613812345678
Slack	U012AB3CD	"张三"	"zhangsan"	—

发送者标签的生成逻辑（用于系统提示词中的对话标识）：

```
// src/channels/sender-label.ts（简化）
export function resolveSenderLabel(params: SenderLabelParams): string | null {
  const display = name ?? username ?? tag ?? "";
  const idPart = e164 ?? id ?? "";

  if (display && idPart && display !== idPart) {
    return `${display} (${idPart})`;    // "张三 (+8613812345678)"
  }
  return display || idPart || null;
}
```

通道特有的目标归一化

每个通道都有自己的目标 ID 格式，需要归一化为统一的内部格式。以 Telegram 和 WhatsApp 为例：

```
// src/channels/plugins/normalize/telegram.ts
export function normalizeTelegramMessagingTarget(raw: string): string | undefined {
  let normalized = raw.trim();

  // 去掉通道前缀
  if (normalized.startsWith("telegram:")) {
    normalized = normalized.slice("telegram:".length).trim();
  } else if (normalized.startsWith("tg:")) {
    normalized = normalized.slice("tg:".length).trim();
  }

  // 解析 t.me 链接
  const tmeMatch = /^https?:\/\/t\.me\/([A-Za-z0-9_]+)$/i.exec(normalized);
  if (tmeMatch?.[1]) {
    normalized = `@${tmeMatch[1]}`;
  }

  return `telegram:${normalized}`.toLowerCase();
}
```

```
// src/channels/plugins/normalize/whatsapp.ts
export function normalizeWhatsAppMessagingTarget(raw: string): string | undefined {
  return normalizeWhatsAppTarget(raw.trim()) ?? undefined;
}
```

归一化后的目标 ID 统一使用 `通道:标识` 格式（如 `telegram:@username` 、 `whatsapp:+8613812345678@s.whatsapp.net` ），使路由和会话管理可以统一处理。

19.2.2 媒体附件处理

附件解析管线

当入站消息包含图片、文件等附件时，OpenClaw 通过 `parseMessageWithAttachments` （位于 `src/gateway/chat-attachments.ts` ）进行解析：

```ts
// src/gateway/chat-attachments.ts（简化）
export type ChatAttachment = {
  type?: string;
  mimeType?: string;
  fileName?: string;
  content?: unknown;      // base64 编码的内容
};

export type ChatImageContent = {
  type: "image";
  data: string;           // base64 数据
  mimeType: string;       // 如 "image/jpeg"
};

export async function parseMessageWithAttachments(
  message: string,
  attachments: ChatAttachment[] | undefined,
  opts?: { maxBytes?: number; log?: AttachmentLog },
): Promise<ParsedMessageWithImages> {
  const maxBytes = opts?.maxBytes ?? 5_000_000;    // 5 MB 上限
  if (!attachments || attachments.length === 0) {
    return { message, images: [] };
  }

  const images: ChatImageContent[] = [];

  for (const [idx, att] of attachments.entries()) {
    const label = att.fileName || att.type || `attachment-${idx + 1}`;
    const b64 = extractBase64(att.content);    // 去掉 data URL 前缀
    validateBase64(b64, label, maxBytes);       // 校验格式和大小

    // MIME 类型嗅探
    const providedMime = normalizeMime(att.mimeType);
    const sniffedMime = normalizeMime(await sniffMimeFromBase64(b64));

    // 只接受图片
    if (sniffedMime && !isImageMime(sniffedMime)) {
      log?.warn(`attachment ${label}: detected non-image (${sniffedMime}), dropping`);
      continue;
    }

    images.push({
      type: "image",
      data: b64,
      mimeType: sniffedMime ?? providedMime ?? att.mimeType,
```

```
    });
  }

  return { message, images };
}
```

这个管线有几个值得注意的设计决策：

1. **大小限制**：默认 5 MB，防止超大文件导致内存溢出。
2. **MIME 类型嗅探**：不信任客户端声称的 MIME 类型，而是从实际 base64 数据的头部字节中检测。这防止了类型伪造攻击。
3. **MIME 不匹配处理**：当嗅探结果与声称的类型不同时，优先使用嗅探结果并记录警告。
4. **仅图片**：当前只支持图片类型的附件直接传递给 AI 模型（因为模型的多模态能力主要支持图片）。非图片附件被丢弃并记录日志。

> **衍生解释：MIME 类型嗅探（MIME Sniffing）**
>
> MIME（Multipurpose Internet Mail Extensions）类型标识文件的格式，如 `image/jpeg` 、 `application/pdf` 。客户端发送的 MIME 类型可能不准确——要么是错误标注，要么是恶意伪造。MIME 嗅探通过读取文件的头部字节（魔数）来检测实际类型。例如 JPEG 文件以 `FF D8 FF` 开头，PNG 文件以 `89 50 4E 47` 开头。OpenClaw 使用 `detectMime` 函数从 base64 数据的前 256 字节中检测实际 MIME 类型。

Data URL 处理

base64 数据可能以 Data URL 格式传入（如 `...` ），需要剥离前缀：

```
// src/gateway/chat-attachments.ts
let b64 = content.trim();
const dataUrlMatch = /^data:[^;]+;base64,(.*)$/.exec(b64);
if (dataUrlMatch) {
  b64 = dataUrlMatch[1];
}
```

base64 校验

解析后的 base64 数据要通过格式和大小校验：

```
// src/gateway/chat-attachments.ts
// 长度必须是 4 的倍数，且只包含合法字符
if (b64.length % 4 !== 0 || /[^A-Za-z0-9+/=]/.test(b64)) {
  throw new Error(`attachment ${label}: invalid base64 content`);
}

const sizeBytes = Buffer.from(b64, "base64").byteLength;
if (sizeBytes <= 0 || sizeBytes > maxBytes) {
  throw new Error(`attachment ${label}: exceeds size limit`);
}
```

> **衍生解释：base64 编码**
>
> base64 是一种将二进制数据编码为 ASCII 字符串的方法。它将每 3 个字节（24 位）分成 4 组 6 位，每组映射到一个可打印字符（A-Z、a-z、0-9、+、/），末尾不足时用 = 填充。因此 base64 字符串的长度必须是 4 的倍数，编码后的大小约为原始数据的 4/3 倍。

19.2.3 消息清洗

信封剥离

OpenClaw 在多 Agent、多通道场景下，消息在转发时会被包裹在"信封"中——前面添加通道和时间戳标记。当这些消息最终到达 AI 模型时，信封需要被剥离，避免干扰模型理解：

```ts
// src/gateway/chat-sanitize.ts
const ENVELOPE_PREFIX = /^\[([^\]]+)\]\s*/;
const ENVELOPE_CHANNELS = [
  "WebChat", "WhatsApp", "Telegram", "Signal", "Slack",
  "Discord", "Google Chat", "iMessage", "Teams", "Matrix",
  "Zalo", "Zalo Personal", "BlueBubbles",
];

export function stripEnvelope(text: string): string {
  const match = text.match(ENVELOPE_PREFIX);
  if (!match) return text;

  const header = match[1] ?? "";
  if (!looksLikeEnvelopeHeader(header)) return text;

  return text.slice(match[0].length);
}
```

信封检测有两个判定条件：

1. **包含时间戳**：如 `[Telegram 2025-02-18T08:30Z] 你好` → 剥离后变为 `你好`
2. **以通道名开头**：如 `[WhatsApp zhangsan] 消息内容` → 剥离后变为 `消息内容`

```ts
// src/gateway/chat-sanitize.ts
function looksLikeEnvelopeHeader(header: string): boolean {
  // ISO 时间戳格式
  if (/\d{4}-\d{2}-\d{2}T\d{2}:\d{2}Z\b/.test(header)) return true;
  // 简化时间戳格式
  if (/\d{4}-\d{2}-\d{2} \d{2}:\d{2}\b/.test(header)) return true;
  // 以已知通道名开头
  return ENVELOPE_CHANNELS.some(label => header.startsWith(`${label} `));
}
```

消息 ID 提示清理

某些消息中可能包含用于回复引用的 `[message_id: ...]` 标注。这些标注对人类读者和 AI 都是噪音，需要清除：

```ts
// src/gateway/chat-sanitize.ts
const MESSAGE_ID_LINE = /^\s*\[message_id:\s*[^\]]+\]\s*$/i;

function stripMessageIdHints(text: string): string {
  if (!text.includes("[message_id:")) return text;    // 快速路径

  const lines = text.split(/\r?\n/);
  const filtered = lines.filter(line => !MESSAGE_ID_LINE.test(line));
  return filtered.length === lines.length ? text : filtered.join("\n");
}
```

批量消息清洗

清洗操作可以应用于整个消息历史数组，且只处理用户角色（`role === "user"`）的消息——助手消息和系统消息不需要清洗：

```ts
// src/gateway/chat-sanitize.ts
export function stripEnvelopeFromMessage(message: unknown): unknown {
  const entry = message as Record<string, unknown>;
  const role = typeof entry.role === "string" ? entry.role.toLowerCase() : "";
  if (role !== "user") return message;    // 只清洗用户消息

  // 处理字符串内容
  if (typeof entry.content === "string") {
    const stripped = stripMessageIdHints(stripEnvelope(entry.content));
    if (stripped !== entry.content) return { ...entry, content: stripped };
  }

  // 处理结构化内容数组（Claude API 的 content block 格式）
  if (Array.isArray(entry.content)) {
    const updated = stripEnvelopeFromContent(entry.content);
    if (updated.changed) return { ...entry, content: updated.content };
  }

  return message;
}
```

这里对 `content` 字段的处理分为两种情况：

- **字符串格式**：直接对整个字符串执行清洗
- **数组格式**：遍历每个 content block，只对 `type === "text"` 的块执行清洗

这是因为 Claude API 支持两种消息格式——简单字符串和结构化 content block 数组。OpenClaw 需要兼容两者。

清洗管线的完整流程

```
入站消息（来自 Telegram/WhatsApp/Discord/...）
    │
    ▼
通道适配器：平台特有格式 → MsgContext
    │
    ├── ChatType 归一化（"private" → "direct"）
    ├── SenderId/SenderName/... 填充
    ├── Body/BodyForAgent 提取
    └── MediaPath/MediaUrl/... 设置
    │
    ▼
附件处理：parseMessageWithAttachments()
    │
    ├── base64 校验和大小检查
    ├── MIME 嗅探验证
    └── 非图片附件丢弃
    │
    ▼
消息清洗：stripEnvelopeFromMessages()
    │
    ├── 信封前缀剥离
    └── message_id 标注清除
    │
    ▼
统一的 MsgContext → 路由 / 会话管理 / AI Agent
```

本节小结

1. **`MsgContext` 统一消息结构**是所有通道的"通用语言"，包含通道标识、发送者身份、消息内容、媒体附件、回复上下文等完整信息。
2. **发送者身份使用多维度表示**（SenderId、SenderName、SenderUsername、SenderE164），适应不同平台的标识体系。E.164 格式用于电话号码标准化。
3. **附件处理管线**执行 base64 校验、MIME 嗅探和大小限制，优先使用嗅探的 MIME 类型而非客户端声称的类型，当前仅支持图片附件直接传递给 AI 模型。
4. **消息清洗**剥离信封前缀和 message_id 标注，且只作用于用户角色的消息，避免干扰 AI 模型的理解。

5. **通道特有的目标归一化**（如 `telegram:@username` 、 `whatsapp:+86...@s.whatsapp.net` ）将异构 ID 格式统一为 `通道:标识` 格式，为下游路由提供一致的寻址基础。